R1.01 - Initiation au Développement : Variables, Types de Données, Allocation Mémoire et Structures Conditionnelles

Thème principal : Initiation au développement logiciel et architecture mémoire

Synthèse exécutive : Ce cours pose les fondations indispensables du développement en BUT 1 Informatique. L'objectif est de comprendre comment un ordinateur manipule la mémoire vive (RAM) à travers le concept de variable, pourquoi le typage rigoureux des données est vital pour éviter les dépassements de capacité (overflow), et comment piloter l'exécution d'un programme via des structures conditionnelles (SI... ALORS... SINON) tout en maîtrisant l'évaluation paresseuse.

Le concept de Variable et le Modèle Mémoire (RAM)

Pour qu'un programme puisse traiter des informations, il doit pouvoir les conserver temporairement en mémoire vive (RAM). Une variable constitue une abstraction de haut niveau d'un emplacement mémoire. Elle se caractérise par trois éléments indissociables : 1) Son nom ou identificateur, utilisé par le développeur dans le code. 2) Son type, qui indique au compilateur la quantité exacte d'octets à réserver et la façon d'interpréter les bits. 3) Sa valeur, qui représente le contenu binaire actuellement stocké. Lors d'une affectation (ex: a = 5), l'ordinateur va à l'adresse mémoire associée à 'a' et y enregistre la valeur binaire correspondante. Si l'on réaffecte 'a = 12', l'ancienne valeur est définitivement effacée.

- Variable : Espace réservé dans la mémoire RAM identifié par une adresse unique et associé à un nom symbolique.
- Typage : Définition précise de la taille en octets et de la nature de la donnée (entier, flottant, caractère, booléen).
- Affectation : Action de stocker une valeur dans la case mémoire d'une variable en écrasant son ancien contenu.

Cas concret enseignant : *Le professeur a utilisé l'analogie du casier dans un vestiaire d'IUT : chaque casier a un numéro fixe (l'adresse mémoire). Vous collez une étiquette avec écrit 'age' (le nom de la variable). La taille du casier détermine ce qu'on peut y mettre (le type). Si vous y mettez un sac rouge (valeur 20), puis plus tard un sac bleu (valeur 21), le sac rouge disparaît. Mettre un objet trop grand provoque un débordement (overflow).*

Les Types Primitifs et la Gestion de la Capacité

Le choix du type est crucial en algorithmique. Un entier 'int' sur 32 bits représente des valeurs de -2 147 483 648 à +2 147 483 647. Dépasser cette limite provoque une erreur d'overflow où le nombre bascule dans les négatifs. Les types flottants ne stockent pas des valeurs exactes mais des approximations (norme IEEE 754), ce qui rend l'utilisation de l'égalité stricte (==) dangereuse entre deux flottants.

- Entiers (int, long) : Nombres sans virgule, signés ou non signés.
- Flottants (float, double) : Nombres à virgule avec précision limitée (norme IEEE 754).
- Booléens (bool) : État logique VRAI (1) ou FAUX (0).
- Conversion de type (Cast) : Transformation explicite ou implicite d'un type vers un autre.

Cas concret enseignant : *L'enseignant a rappelé le bogue de la fusée Ariane 5 en 1996 : la conversion d'un flottant 64 bits en un entier 16 bits a provoqué un overflow, entraînant la destruction de la fusée. Il a conseillé de ne jamais utiliser 'float' pour des montants financiers, mais plutôt des entiers représentant des centimes.*

Structures Conditionnelles et Évaluation Logique

Par défaut, les instructions s'exécutent ligne par ligne. Les structures conditionnelles permettent d'introduire des choix. Le bloc SI évalue une expression booléenne. L'évaluation paresseuse est fondamentale : dans '(A && B)', si A est FAUX, B n'est pas évalué car le résultat global est nécessairement FAUX.

- Structure SI / SINON SI / SINON : Branchement conditionnel modifiant le flux d'exécution séquentiel.

• Opérateurs Logiques : ET (&&), OU (||), NON (!).

• Évaluation paresseuse (Short-circuit evaluation) : Arrêt de l'évaluation dès que le résultat global d'une expression logique est garanti.

Cas concret enseignant : *Exemple d'un contrôle d'accès : 'SI (utilisateur_existe ET mot_de_passe_correct)'. Si 'utilisateur_existe' est FAUX, le système ne vérifie même pas le mot de passe, ce qui évite des plantages (comme la lecture d'une variable nulle) et optimise les performances.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi 0.1 + 0.2 n'est pas exactement égal à 0.3 quand on l'affiche en programmation ?
Réponse du professeur : C'est dû à la représentation en base 2 des nombres à virgule flottante (norme IEEE 754). Certains nombres décimaux finis en base 10 ont une écriture infinie en base 2, ce qui génère de légères erreurs d'arrondi.

Question étudiant : Est-ce qu'une variable déclarée dans un bloc SI existe toujours après la fin du bloc SINON ?
Réponse du professeur : Cela dépend de la portée (scope) de la variable et du langage utilisé (par exemple en C ou Java, les accolades définissent un bloc de visibilité et la variable meurt à la fin du bloc).

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme d'un programme qui demande l'âge d'un utilisateur et son statut d'étudiant (booléen), puis calcule le prix d'un billet de cinéma sachant que : tarif plein = 10 €, mineurs (< 18 ans) = 6 €, étudiants majeurs = 7.50 €, seniors (>= 65 ans) = 5 €.

Résolution pas à pas :
Étape 1 : Déclarer les variables 'age' (Entier), 'estEtudiant' (Booléen), 'prix' (Flottant). Étape 2 : Saisir les valeurs d'entrée. Étape 3 : Évaluer les conditions dans un ordre logique (du plus restrictif/prioritaire au tarif général) : - SI (age < 18) ALORS prix = 6.00 - SINON SI (age >= 65) ALORS prix = 5.00 - SINON SI (estEtudiant == VRAI) ALORS prix = 7.50 - SINON prix = 10.00 Étape 4 : Afficher le prix.

Résultat : Pour un utilisateur de 20 ans avec un statut étudiant à VRAI, le programme évalue les conditions et attribue le tarif de 7.50 €. L'ordre des conditions garantit par ailleurs qu'un mineur étudiant paiera bien le tarif mineur (6 €), qui est le plus avantageux.

■ Auto-Évaluation Flash d'Examen

Q : Que se produit-il si l'on tente d'affecter la valeur 300 à une variable entière stockée sur un octet non signé (uint8_t) ?

R : Un dépassement de capacité (overflow) se produit, car la valeur maximale stockable sur 1 octet non signé est 255.

Q : Dans une condition 'SI (A || B)', quelle est l'utilité de l'évaluation court-circuit si A est VRAI ?

R : L'ordinateur n'évalue pas B car le résultat global de l'expression OU est garanti d'être VRAI dès que A est VRAI.

Q : Quelle est l'erreur principale à éviter lors de la comparaison de deux variables de type float ?

R : Éviter d'utiliser l'opérateur d'égalité stricte (==) en raison des approximations de calculs binaires sur les nombres à virgule.